Table of Contents

List of Figures

List of Tables

Abstract

1 Introduction 1

1.1 About (Introduction to Lecture Comprehension Analysis) 2

1.2 Problem Statement 2

1.3 Purpose of the Project 3

1.4 Objectives of the Project 3

1.5 Scope of the Project 4

1.6 Organization of the Report 4

1.7 Chapter Summary 5

2 Literature Survey 6

2.1 Introduction 7

2.2 Literature Review 7

2.2.1 LectureBank: Learning What to Learn First (Li et al., 2019) 7

2.2.2 Cross-Domain Prerequisite Chain Learning (Li et al., 2021) 7

2.2.3 Prerequisite Relation Learning for Concepts in MOOCs (Pan et al., 2017) 7

2.2.4 Knowledge Tracing: Bayesian and Deep Approaches 8

2.2.5 Whisper: Robust Speech Recognition (Radford et al., 2023) 8

2.2.6 CLIP and Visual Understanding of Slides (Radford et al., 2021) 8

2.2.7 Sentence-BERT for Semantic Similarity (Reimers and Gurevych, 2019) 8

2.2.8 NetworkX for Graph Analysis (Hagberg et al., 2008) 8

2.2.9 Commercial AI Study Tools 8

2.3 Summary of Literature Survey 9

2.4 Research Gap Identified 10

2.5 Chapter Summary 10

3 Project Overview 11

3.1 Project Deliverables 12

3.2 Timeline with Milestones 13

3.3 Risk Analysis and Mitigation 14

3.4 Chapter Summary 14

4 Methodology 15
4.1 Proposed System Overview 16
4.2 Existing System Architecture 18
4.3 Proposed System Architecture 20
5 Project Design & Process Workflow 21
5.1 System Flowchart Diagram 22
5.2 Use Case Diagram 23
5.3 Algorithms 23
5.3.1 Transcription 24
5.3.2 Lecture-Structure Extraction 24
5.3.3 Prerequisite Classification 25
5.3.4 Graph Construction 25
5.3.5 Quiz Generation and Remediation 25
5.3.6 Graph Refinement by Weak Supervision 27
5.4 Software Specifications 28
5.5 Hardware Specifications 29
5.6 User Requirements 29
5.7 Chapter Summary 30
6 Conclusion and Future Scope 31
Conclusion 32
Limitations 32
Future Scope 4
Final Remarks 33
References 34
Publications 35

List of Figures

Fig. 4.1 Agile Development 16
Fig. 4.2 Existing System Architecture Flowchart 18
Fig. 4.3 Existing System Architecture 19
Fig. 4.4 System Architecture 20
Fig. 5.1 System Flowchart 22
Fig. 5.2 Use Case Diagram 23
Fig. 5.3 Concept Extraction Flowchart 24
Fig. 5.4 Quiz and Remediation Flowchart 26
Fig. 5.5 Refinement Loop Flowchart 27

List of Tables

Table 1.1 Mapping of Objectives to Report Chapters 5
Table 2.1 Summary of Literature Survey 9
Table 3.1 Timeline of the Project 13
Table 3.2 Risks and Mitigation 14
Table 4.1 Comparison of Existing Systems and LecGap 19
Table 5.1 Example of Remediation Order 27
Table 5.2 Software Requirements 28
Table 5.3 Hardware Requirements 29

Abstract

Recorded lectures are now a primary mode of learning, yet they cannot tell a student whether a topic has actually been understood. When a learner fails a question, the cause is often a weak prerequisite concept taught earlier, and neither the video nor existing AI study tools can locate it. Instructors face a similar blind spot, since classroom-wide confusion is invisible to them. The Lecture Comprehension Gap Detector (LecGap) addresses this problem with an end-to-end pipeline. Lecture audio is transcribed with Whisper, a language model extracts the concepts that are taught together with the passages that explain them, and a prerequisite classifier and transcript evidence are combined into a course-level directed acyclic graph. Students take a short, server-graded quiz; each wrong answer is traced through the graph to the root-cause prerequisite, and short video clips are served in dependency order. A co-failure rule refines the graph from student responses, and instructors receive a confusion heatmap and a comparison of taught order against learned order. The prerequisite classifier achieves an F1 score of 0.569 on LectureBank, and in simulation the refinement step raises graph F1 from 0.571 to 0.857. The system has been run end to end on real lectures of up to 86 minutes, and is implemented with FastAPI and Streamlit with live job progress.

Keywords: Lecture Comprehension, Prerequisite Learning, Concept Graph, Knowledge Gap Detection, Speech Recognition, Large Language Models, Educational Data Mining, Personalised Remediation, Weak Supervision, Instructor Analytics

CHAPTER 1

INTRODUCTION

1.1 About (Introduction to Lecture Comprehension Analysis):

Recorded lectures have become one of the most common ways in which engineering students learn. Platforms such as NPTEL and YouTube make thousands of hours of high-quality teaching freely available, and students can pause, rewind and revisit any part of a lecture. However, a video can only be played back; it cannot tell a student whether they have actually understood it. When a learner struggles with a topic, the real cause is often not the topic itself but a weak or missing prerequisite concept that was taught earlier, and neither the video nor the student can easily locate it.

The same difficulty affects instructors. A teacher who uploads a lecture has no direct way of knowing which concepts confused the class, or whether the order in which concepts were taught matches the order in which they depend on one another. Existing AI study tools can summarise a lecture or generate quiz questions, but they treat every question independently and do not model how concepts build upon each other.

LecGap (Lecture Comprehension Gap Detector) is an AI-based system designed to close this gap. It converts a lecture recording into a timestamped transcript, extracts the concepts that were taught, and organises them into a course-level concept graph in which an edge means that one concept is a prerequisite of another. A student then answers a short quiz; when an answer is wrong, the system traverses the graph to find the underlying weak prerequisite and recommends short video clips in the correct learning order. For instructors, the same data is summarised as a confusion heatmap and as a comparison between the order in which concepts were taught and the order in which they should be learned.

The importance of concept dependencies is easy to see in a technical subject. A student cannot meaningfully follow a lecture on gradient descent without understanding loss functions, and cannot follow backpropagation without gradient descent. When the lecture is delivered as a single linear video, these dependencies are left implicit; the instructor mentions them in passing, and the student is expected to notice and remember them. LecGap makes these dependencies explicit and uses them as the basis for diagnosis.

LecGap has been designed as a practical tool rather than a laboratory prototype. It runs on ordinary hardware by relying on hosted speech and language services, it handles recordings of more than eighty minutes, and it reports live progress while it works.

1.2 Problem Statement:

Video lectures are widely used, yet they provide no mechanism for diagnosing why a student fails to understand a topic. A student who answers a question incorrectly is usually told only that the

answer is wrong. The tools available today do not identify which earlier concept is responsible for the failure, and they do not tell the student what to revisit first. Instructors face a related problem: classroom-wide confusion is invisible to them unless students raise it explicitly.

Existing research on prerequisite learning is largely offline. Prerequisite models are trained once on a fixed benchmark and are never corrected using data from real students. Systems that do track student knowledge, such as knowledge-tracing models, work from quiz history alone and do not know the content of the lecture itself. The major problems that motivate this project are:

- Students can watch an entire lecture without learning which concepts they failed to grasp
- Quiz tools mark answers right or wrong but cannot trace a failure back to its root-cause prerequisite
- Learners are shown their mistakes, but not the order in which concepts should be revised
- Instructors have no class-level view of confusion and cannot check whether the teaching order respects concept dependencies
- Prerequisite models are static and do not improve from the behaviour of the students who use them

Therefore, there is a need for a single system that can automatically build a concept graph from a lecture, diagnose the root cause of a student's mistake, personalise the remediation, and give the instructor a classroom-level diagnostic. LecGap is proposed to meet this need.

1.3 Purpose of the Project:

The purpose of LecGap is to make recorded lectures diagnostic rather than passive. By linking every quiz result to the concept graph of the lecture, the system turns a wrong answer into an actionable recommendation: which prerequisite to revise, in what order, and with which short video clip. This reduces the time a student spends searching through a long recording and helps them repair the actual gap in understanding instead of repeating the same lecture.

A second purpose is to give instructors evidence about their own teaching. When many students fail on the same concept, or when the sequence in which topics were taught differs from the sequence suggested by the prerequisite graph, the instructor can see this immediately and adjust the lecture. Finally, the project aims to demonstrate that a prerequisite model need not remain fixed: signals from real student performance can be used to strengthen correct edges and weaken spurious ones, so that the graph improves with use.

1.4 Objectives of the Project:

The primary objective of the project is to develop an end-to-end system that detects comprehension gaps in recorded lectures and guides students towards the exact material that closes them. The system is designed to work with real lecture recordings of varying length and to remain usable on modest hardware by relying on hosted inference where heavy computation is required. The specific objectives are:

- To extract timestamped concepts from lecture audio and visual content using speech recognition and language models
- To classify prerequisite relationships between concepts and benchmark the classifier on the LectureBank dataset
- To build a per-course concept graph (a directed acyclic graph) with duplicate merging and cycle resolution
- To generate quizzes and personalise remediation by tracing wrong answers through the graph and serving short video clips in dependency order
- To refine the prerequisite graph continuously from student performance using weak supervision
- To give instructors a confusion heatmap and a taught-versus-learned order comparison for their lectures

1.5 Scope of the Project:

The scope of the project covers the complete lifecycle of a lecture: uploading the recording, transcribing it, extracting concepts, building the graph, cutting concept-specific clips, generating and grading a quiz, producing a remediation plan, and presenting analytics to the instructor. The current implementation is a working baseline built with a Streamlit front end and a FastAPI back end, with background jobs that report live progress to the user.

The project has been developed and tested on technical lecture recordings in machine learning, deep learning and probability, ranging from a few minutes to more than eighty minutes in length. The prerequisite classifier is benchmarked on LectureBank, and the refinement mechanism is validated using simulated student data. The following are outside the present scope: deployment to a large number of concurrent users, support for languages other than English, and evaluation on a live classroom over a full semester. These are identified as future work in Chapter 6.

1.6 Organization of the Report:

The remainder of this report is organised into five chapters, which are summarised below.

- **Chapter 2 (Literature Survey):** reviews prerequisite learning, knowledge tracing, speech and vision models, and existing study tools, and summarises their limitations
- **Chapter 3 (Project Overview):** lists the deliverables of the project and the timeline with milestones
- **Chapter 4 (Methodology):** describes the development approach, the existing systems and the proposed layered architecture
- **Chapter 5 (Project Design and Process Workflow):** presents the system flowchart, the use cases, the algorithms, and the software, hardware and user requirements
- **Chapter 6 (Conclusion and Future Scope):** summarises the results achieved and the work planned for the next semester

Table 1.1 shows where each objective listed in Section 1.4 is addressed in this report and what the outcome was.

Objective	Addressed in	Outcome
Extract timestamped concepts from lecture audio and visuals	Sections 4.1, 5.3.1 and 5.3.2	Whisper transcripts and structured concept extraction
Classify prerequisites and benchmark the classifier	Section 5.3.3	Frozen-encoder classifier with F1 of 0.569 on LectureBank
Build a per-course concept graph	Section 5.3.4	Directed acyclic graph with merging, cycle removal and learning order
Generate quizzes and personalise remediation	Section 5.3.5	Server-graded quiz and clips ordered by prerequisite
Refine the graph from student performance	Section 5.3.6	Co-failure rule; F1 from 0.571 to 0.857 in simulation
Give instructors a diagnostic view	Sections 3.1 and 5.2	Confusion heatmap and taught-versus-learned divergence

Table 1.1: Mapping of Objectives to Report Chapters

The chapters follow the order of the project itself: the problem and its background first, then the design, and finally the results and the next steps.

1.7 Chapter Summary:

This chapter introduced the problem that LecGap addresses and set out the scope of the work. The main points are summarised below.

- **Problem:** recorded lectures cannot diagnose why a student fails, and instructors cannot see classroom-wide confusion
- **Proposal:** LecGap builds a concept graph from each lecture, traces wrong answers to root-cause prerequisites and serves short clips in dependency order
- **Objectives:** extract concepts, classify prerequisites, build the graph, personalise remediation, refine the graph and support instructors
- **Scope:** a working baseline for technical lectures, with evaluation in a live classroom left for future work

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction:

The literature relevant to LecGap lies at the intersection of four areas: prerequisite relation learning for educational concepts, student knowledge modelling, speech and vision models for processing lecture content, and graph-based representations of knowledge. This chapter reviews representative work in each area, describes what each contributes to the project, and identifies the limitation that LecGap is designed to address. The chapter ends with a summary table that compares the surveyed work.

2.2 Literature Review:

2.2.1 LectureBank: Learning What to Learn First (Li et al., 2019)

Li, Fabbri, Tung and Radev introduced LectureBank, a collection of lecture slides in natural language processing together with a set of 208 topics annotated with prerequisite relations. The work frames prerequisite chain learning as the task of deciding, for a pair of concepts, whether one must be understood before the other, and it provides baseline classifiers for this task. LectureBank is the canonical benchmark for the problem and is used in LecGap to train and evaluate the prerequisite classifier. Its limitation is that the annotations are fixed and derive from slides rather than from spoken lectures, so a model trained on it alone cannot adapt to a new course or correct itself from student behaviour. In LecGap the benchmark serves two purposes: it provides the training data for the classifier and the yardstick against which every fine-tuned variant is compared.

2.2.2 Cross-Domain Prerequisite Chain Learning (Li et al., 2021)

Li, Yan and Radev extended prerequisite learning to the setting in which labelled data exists for one subject but not for another. Their method uses variational graph autoencoders to transfer knowledge about prerequisite structure across domains without target-domain labels. The work shows that prerequisite structure is partly transferable between subjects, which supports the idea of reusing a classifier trained on LectureBank for lectures in related subjects. The approach, however, is evaluated offline on benchmark data and does not use any feedback from learners.

2.2.3 Prerequisite Relation Learning for Concepts in MOOCs (Pan et al., 2017)

Pan and colleagues studied prerequisite relations among concepts in massive open online courses. They combined features derived from the course content, such as how concepts are referenced across videos, with embedding-based features to predict dependencies between concepts. This work established that signals from lecture material can be used to infer prerequisites. It operates on concepts that are already identified, whereas LecGap must also extract the concepts automatically from raw lecture audio. The study is nevertheless relevant because it shows that evidence inside the

lecture material carries a useful prerequisite signal, an idea LecGap exploits by storing the instructor's own spoken links as evidence edges.

2.2.4 Knowledge Tracing: Bayesian and Deep Approaches

Bayesian Knowledge Tracing models a student's mastery of a skill as a hidden state that is updated after every response, and Deep Knowledge Tracing, proposed by Piech and colleagues, replaces it with a recurrent network that predicts the next response from the full history. Both predict whether a student will answer correctly, but they take skills as given and know nothing about lecture content, so they cannot say which earlier concept to revisit.

2.2.5 Whisper: Robust Speech Recognition (Radford et al., 2023)

Whisper is a speech recognition model trained on a very large amount of weakly supervised multilingual audio. It produces accurate, timestamped transcripts and is robust to accents and background noise, which makes it suitable for real lecture recordings. In LecGap, Whisper is the first stage of the pipeline, and its segment-level timestamps are essential because they allow every extracted concept and every video clip to be tied to an exact position in the lecture.

2.2.6 CLIP and Visual Understanding of Slides (Radford et al., 2021)

CLIP learns a joint representation of images and text from natural-language supervision. When combined with optical character recognition, it can describe what is shown on a slide or whiteboard and can help detect when the visual content changes. Lecture content is often presented visually as well as verbally, so LecGap uses this family of models to capture concepts that are shown but not spoken. The visual track is the less mature part of the current implementation and is planned for further development.

2.2.7 Sentence-BERT for Semantic Similarity (Reimers and Gurevych, 2019)

Sentence-BERT adapts transformer language models to produce fixed-size sentence embeddings that can be compared efficiently using cosine similarity. LecGap uses a compact Sentence-BERT-style encoder to embed concept names, to merge duplicate concepts across lectures, and as the feature extractor of the prerequisite classifier. The compact model keeps inference fast enough to run on an ordinary CPU.

2.2.8 NetworkX for Graph Analysis (Hagberg et al., 2008)

NetworkX is a Python library for creating and analysing graphs. It provides directed graphs, cycle detection and topological sorting, which are exactly the operations needed to represent prerequisites as a directed acyclic graph and to obtain a valid learning order. LecGap relies on it to merge nodes, break cycles using edge confidence, and compute the order in which concepts should be revised. Because a course graph is small, with tens to a few hundred nodes, these operations run instantly and need no specialised graph database.

2.2.9 Commercial AI Study Tools

Tools such as YouLearn, Knowt and NotebookLM can summarise recorded content and generate flashcards or quiz questions. They make studying faster but treat each question in isolation. None of them builds an explicit prerequisite structure for a course, traces a wrong answer to its root cause, or provides instructors with a class-level view of confusion. This is the specific gap that LecGap addresses.

2.3 Summary of Literature Survey:

Table 2.1 summarises the surveyed work. A common pattern emerges: existing research either learns prerequisite structure offline from fixed data, or models student knowledge without reference to lecture content, and existing tools generate questions without diagnosing root causes. No surveyed system combines automatic concept extraction from lectures, a self-correcting prerequisite graph, root-cause remediation and instructor analytics in a single pipeline.

Sr. No.	Work (Author, Year)	Method	Key Contribution	Limitation
1	Li et al., 2019 (LectureBank)	Annotated slides; prerequisite classification baselines	Canonical benchmark of 208 topics with prerequisite labels	Fixed labels; no adaptation to new courses or learners
2	Li et al., 2021	Variational graph autoencoders	Cross-domain transfer of prerequisite structure	Offline; no learner feedback
3	Pan et al., 2017	Content and embedding features on MOOCs	Prerequisites inferred from course material	Assumes concepts are already given
4	Corbett and Anderson, 1994	Bayesian Knowledge Tracing	Mastery modelled as a hidden state	Skills treated as independent; ignores lecture content
5	Piech et al., 2015	Recurrent network (Deep Knowledge Tracing)	Predicts next response from history	Does not identify which concept to revisit
6	Radford et al., 2023 (Whisper)	Weakly supervised speech recognition	Accurate timestamped transcripts	Produces text only; no concept understanding
7	Radford et al., 2021 (CLIP)	Joint image-text representation	Describes visual content of slides	Needs a concept layer to be educationally useful
8	Reimers and Gurevych, 2019	Sentence-BERT embeddings	Fast semantic similarity	A building block, not a diagnostic system
9	YouLearn, Knowt, NotebookLM	LLM-based study assistants	Fast summaries and quizzes	No prerequisite model or instructor analytics

Table 2.1: Summary of Literature Survey

Three observations follow from Table 2.1. First, prerequisite research and knowledge-tracing research have developed separately, so no existing work connects what a student gets wrong to the structure of the lecture in which the concept was taught. Second, the speech and vision models surveyed are mature enough to be used as building blocks, which allows this project to concentrate on the concept layer above them. Third, none of the surveyed systems closes the loop by using student responses to correct the prerequisite model, which motivates the refinement stage of LecGap and set out in Chapter 1.

2.4 Research Gap Identified:

The survey leads to the following gaps in existing work, each of which corresponds to a component of LecGap:

- **No end-to-end pipeline:** speech recognition, concept extraction, prerequisite modelling, remediation and instructor analytics have been studied separately but never combined for lecture video
- **Static prerequisite models:** models are trained once on fixed benchmarks and are never corrected using the behaviour of real students
- **Concept-blind quizzes:** study tools grade answers but cannot trace a wrong answer back to the earlier concept that caused it
- **No instructor feedback:** teachers cannot see classroom-wide confusion, or whether their teaching order respects concept dependencies

LecGap addresses these gaps by building the concept graph directly from the lecture, linking quiz results to that graph, refining the graph from student responses, and presenting the outcome to both students and instructors. The next chapter describes the deliverables and the timeline through which this is achieved.

2.5 Chapter Summary:

This chapter reviewed the research and tools on which LecGap builds. The key conclusions are:

- **Prerequisite learning:** LectureBank and later work show that prerequisite relations can be learned, but only offline and from fixed data
- **Knowledge tracing:** Bayesian and deep models predict student mastery but know nothing about lecture content
- **Building blocks:** Whisper, CLIP, Sentence-BERT and NetworkX provide mature components for transcription, visual description, similarity and graph handling

CHAPTER 3

PROJECT OVERVIEW

3.1 Project Deliverables:

The project is organised around a set of concrete deliverables, each of which corresponds to one stage of the LecGap pipeline or to a supporting component. The deliverables of the current baseline are listed below.

- **Transcription module:** converts lecture audio into a timestamped transcript using Whisper, with chunking and retry handling for long recordings
- **Concept extraction module:** a lecture-structure pass that reads the transcript in windows, identifies the passages in which each concept is taught, and records the spoken links between concepts as evidence
- **Prerequisite classifier:** a frozen sentence encoder with a logistic classifier, benchmarked on LectureBank, used for concept pairs not covered by transcript evidence
- **Concept graph builder:** a directed acyclic graph with duplicate merging, confidence-weighted edge merging, cycle resolution and a topological learning order
- **Clip segmentation module:** FFmpeg-based cutting of short concept-specific clips with parallel processing
- **Quiz and remediation module:** multiple-choice questions generated from the lecture's own sentences, server-side grading, and remediation ordered by prerequisite dependencies
- **Refinement module:** a co-failure rule that strengthens or weakens graph edges from student responses, validated with simulated students
- **Student and faculty dashboards:** quiz and remediation views for students, and confusion heatmap, taught-versus-learned divergence, concept graph and lecture timeline for instructors
- **Evaluation scripts:** reproducible scripts for classifier benchmarking and for the refinement experiment, so that every reported number can be regenerated
- **Backend services:** a FastAPI application with a durable job registry, live progress streaming, and an automated test suite

Together these deliverables form a complete working baseline: a lecture can be uploaded at one end and, at the other end, a student receives a remediation plan and an instructor receives a diagnostic view. Each deliverable has automated tests, so that later changes to one module do not silently break the others.

In addition to the software, the project delivers technical documentation that describes every module, the API surface and the known gaps, together with the review presentations and this report. The Streamlit baseline has been documented in detail so that it can serve as a reference while the React front end is built.

3.2 Timeline with Milestones:

The project follows an agile approach in which each phase ends with a runnable result. Table 3.1 records the milestones completed during Semester VII and the work planned for Semester VIII.

Period	Activity	Deliverable / Outcome	Status
Up to Aug 2026	Topic selection, planning, architecture and documentation	Project plan, system design and Review 1	Completed
Aug 2026	Environment setup, Whisper transcription, LLM access layer with caching	Working end-to-end thin slice	Completed
Aug 2026	Concept extraction; LectureBank labels acquired	Spoken-track extraction; 208-topic benchmark ready	Completed
Aug 2026	Classifier benchmarking and Kaggle GPU fine-tuning	Frozen-encoder baseline (F1 0.569) adopted	Completed
Aug – Sep 2026	Concept graph, clip segmentation, quiz loop, remediation ordering	DAG builder, FFmpeg clips, graded quiz	Completed
Sep 2026	Refinement experiment and faculty dashboard	F1 0.571 to 0.857 on simulated students; heatmap and divergence views	Completed
Sep 2026	Full-length lecture runs, server-graded quizzes, interactive graph	End-to-end runs on lectures of 21 to 86 minutes	Completed
Sep 2026	Job registry with live progress, architecture refactor, dark-theme interface	Non-blocking jobs; more than 700 automated tests	Completed
Semester VIII	React front-end rebuild and job-queue hardening	Responsive interface and stable queues	Planned
Semester VIII	Wider evaluation, final testing and report	Final system and project report	Planned

Table 3.1: Timeline of the Project

Three technical decisions shaped the timeline and are worth recording:

- **Classifier choice:** several fine-tuned cross-encoders were tried, but none beat the frozen-encoder baseline, so the simpler model was adopted and further fine-tuning was set aside
- **Transcript-first prerequisites:** prerequisites stated explicitly by the instructor are stored as evidence edges, and the classifier is used only for the remaining pairs
- **Non-blocking jobs:** long operations were moved into a job registry with live progress after early versions blocked the interface during transcription and extraction

These decisions explain why the work after August concentrated on the quiz loop, refinement and reliability, and they define the starting point for Semester VIII, where the emphasis moves from building the pipeline to evaluating it.

3.3 Risk Analysis and Mitigation:

Several risks were identified early and managed during Semester VII. Table 3.2 lists the main risks, their impact and the mitigation that was adopted.

Risk	Impact	Mitigation
Hosted speech or language service is slow or unavailable	Processing stalls or fails	Response caching, retry with back-off, and durable jobs that can be resumed
Language model returns malformed or truncated output	Concepts or passages are lost	Repair and salvage of partial output before the result is stored
Fine-tuned prerequisite models do not beat the baseline	Wasted effort and lower accuracy	Shipping gate: the frozen-encoder baseline is used unless a model clearly improves on it
Long recordings block the interface	Users cannot tell whether work is running	Background job registry with live progress and stage reporting
No real classroom data is available yet	Refinement cannot be fully validated	Validation with simulated students now; live evaluation planned for Semester VIII
Uploads or answer keys are misused	Quiz integrity and data security are weakened	Server-side grading, size-limited uploads, sanitised file names and an optional API key

Table 3.2: Risks and Mitigation

These mitigations explain why reliability work, rather than new features, occupied a large part of September, and they remain in force for the evaluation planned in Semester VIII.

3.4 Chapter Summary:

This chapter described what the project delivers and how the work was organised in time. The main points are:

- **Deliverables:** transcription, concept extraction, a prerequisite classifier, graph building, clip cutting, quizzes with remediation, refinement, two dashboards and backend services
- **Progress:** all planned Semester VII milestones were completed between August and September 2026
- **Decisions:** a frozen-encoder classifier, transcript-first prerequisites and non-blocking jobs shaped the work
- **Next:** Semester VIII focuses on the React front end, job-queue hardening, wider evaluation and the final report

CHAPTER 4

METHODOLOGY

4.1 Proposed System Overview:

The methodology adopted for LecGap is an agile, iterative one. The system is built in small vertical slices, each of which runs end to end before the next is started: first a thin slice from upload to transcript, then concept extraction, then the prerequisite classifier and graph, then clips, quizzes and refinement, and finally the dashboards. After every slice the result is tested on a real lecture and the findings feed into the next iteration. This approach was chosen because several design decisions, such as which prerequisite model to adopt, could only be made after measuring real results. Each increment was demonstrated on a real lecture recording before the next was started. Short feedback cycles also kept the design honest: when a stage did not perform as expected, the effort moved to the next-best option instead of being sunk into one approach.

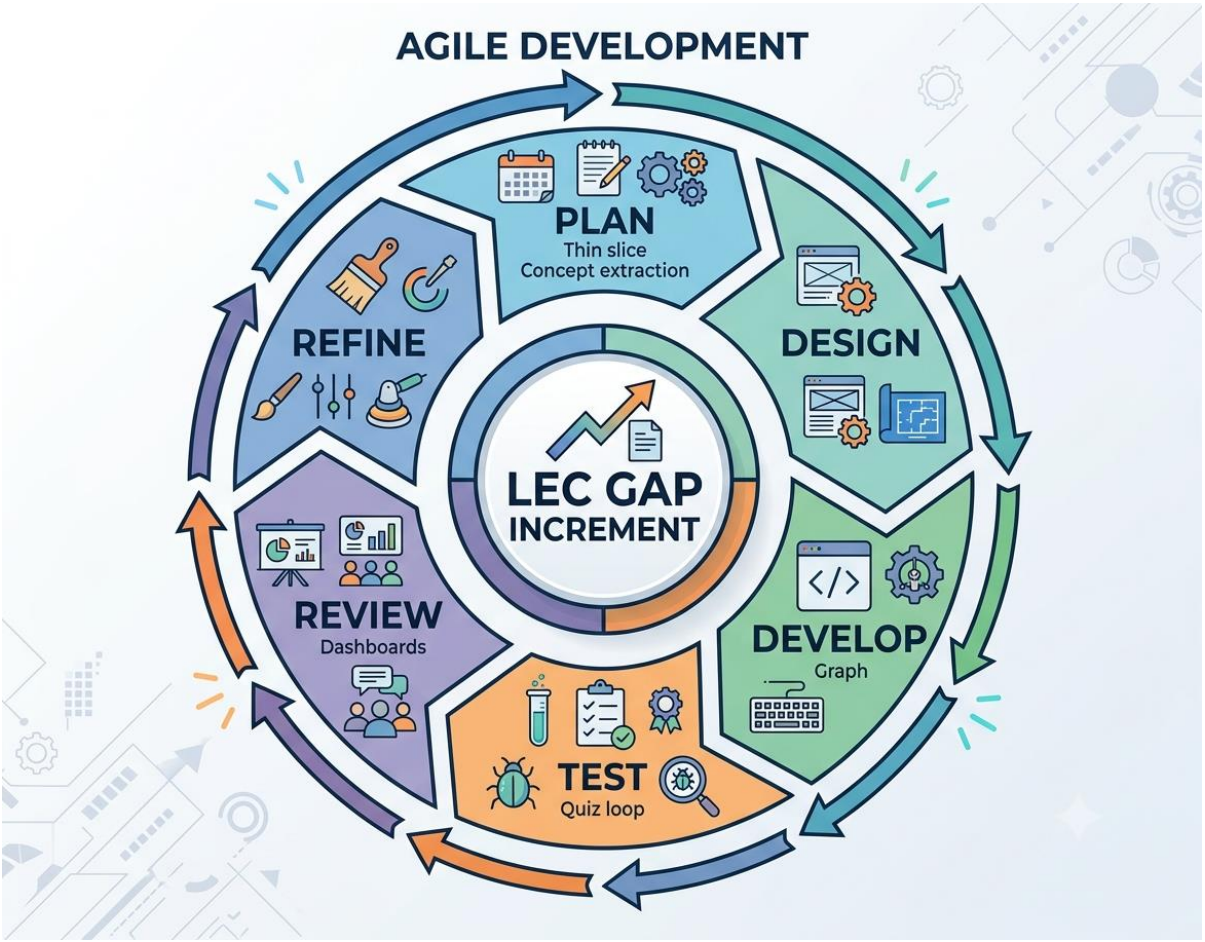


Fig. 4.1: Agile Development

The proposed system takes a lecture recording as input and produces three outputs: a concept graph of the course, a personalised remediation plan for each student, and a diagnostic view for the instructor. The system is divided into eight logical stages, described below.

- **Speech to text:** Whisper transcribes the audio into short timestamped segments
- **Finding what is taught:** a language model reads the transcript in overlapping windows and identifies the concepts and the passages in which they are taught
- **Finding what must be learned first:** concept pairs are scored by a prerequisite classifier, and transcript evidence is used wherever the instructor states a dependency explicitly
- **Drawing the map:** concepts and edges are merged into a directed acyclic graph with a valid learning order
- **Cutting the video:** FFmpeg produces a short clip for each concept
- **Quiz and personalised fix:** a quiz is generated and graded, and wrong answers are traced through the graph to the weakest prerequisite
- **Getting smarter over time:** student responses refine the graph through a co-failure rule
- **Helping the professor:** results are aggregated into a confusion heatmap and a taught-versus-learned comparison

The development process relies on a small number of consistent practices. All source code is kept under version control, and every pipeline stage has automated tests, so that a change to one module can be checked against the others before it is accepted. Experiments that need a GPU, such as classifier fine-tuning, are run in cloud notebooks and only their results are brought back into the main code base.

Evaluation is built into the method rather than added at the end. The prerequisite classifier is compared on a fixed split of LectureBank using precision, recall and F1, and a candidate model is accepted only if it improves on the current baseline. The refinement step is evaluated against a reference graph with the same metrics, and the end-to-end pipeline is checked by running complete lectures of different lengths and recording the time taken by each stage.

Design decisions and known gaps are recorded in written documentation as the work proceeds. This habit made it possible to review the baseline systematically at the end of the semester, to list what remains to be done, and to hand the plan for Semester VIII to every member of the team without relying on memory.

4.2 Existing System Architecture:

At present a student who does not understand part of a lecture has three options. The first is to rewatch the recording and hope to notice the problem, which is slow and unguided. The second is

to use a general-purpose AI study tool that summarises the content or generates questions; these tools are fast, but each question is independent and the tool has no model of how concepts depend on one another. The third is to ask the instructor, which does not scale to a large class. The existing flow is shown in Fig. 4.2.

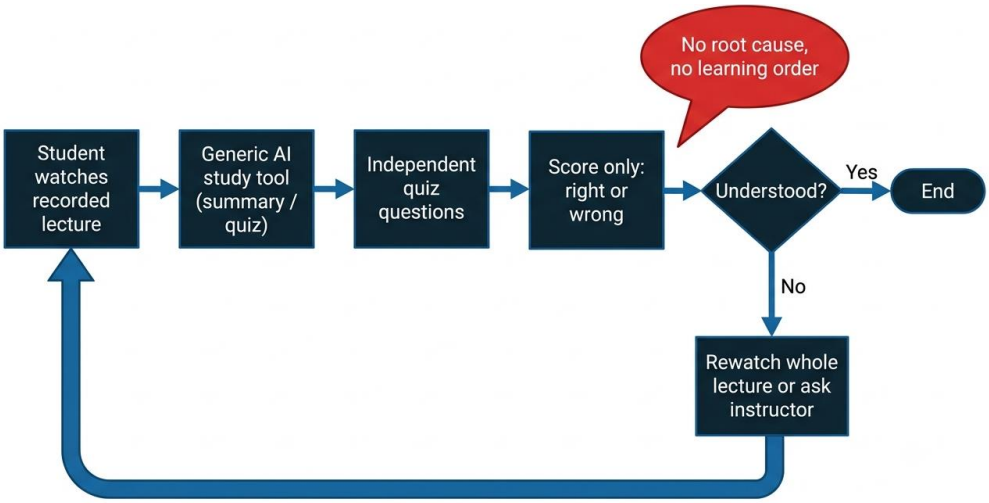


Fig. 4.2: Existing System Architecture Flowchart

On the research side, existing prerequisite-learning systems follow a fixed offline pipeline: a model is trained on a labelled benchmark, evaluated once, and then used unchanged, so an edge that is wrong for one course stays wrong until the next offline retraining. There is no feedback path from students to the model, no link to the spoken content of a particular lecture, and no analytics for instructors. The structure of such systems is shown in Fig. 4.3.

The main weaknesses of the existing architecture are the absence of an explicit prerequisite structure, the lack of root-cause diagnosis, the absence of any learning from real students, and the lack of classroom-level insight for the instructor. Taken together, these weaknesses mean that students receive information about what they got wrong but no guidance about what to do next.

A further weakness is that existing tools work on the final text of a lecture rather than on its timeline. Because they do not know where in the recording each idea was taught, they cannot point a student to the exact few minutes that explain a missing concept, and a student who is told to “revise the lecture” must still search through the whole recording. LecGap keeps every concept anchored to the time span in which it is taught so that remediation can be delivered as short clips. It is also why plain quiz scores cannot tell an instructor whether a low result reflects a difficult concept or a missed prerequisite. A system anchored to the timeline can instead return a short clip for a single missing idea.

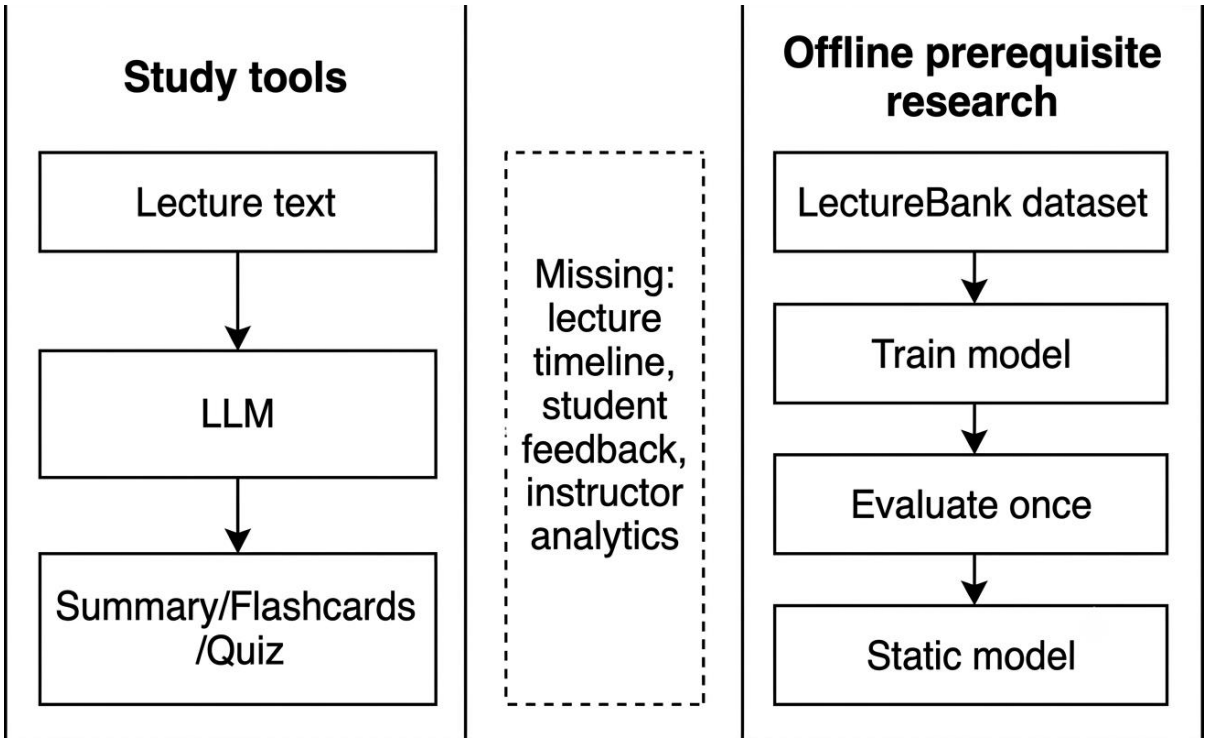


Fig. 4.3: Existing System Architecture

Table 4.1 compares the existing approaches with the proposed system on the features that matter most for lecture comprehension.

Feature	Study tools	Offline prerequisite research	LecGap
Concept extraction from the lecture	Text summary only	Concepts assumed to be given	Automatic and timestamped
Explicit prerequisite graph	No	Yes, trained offline	Yes, built per course
Root-cause diagnosis of wrong answers	No	No	Yes, by graph traversal
Short clips for remediation	No	No	Yes, in dependency order
Learning from student responses	No	No	Yes, co-failure refinement
Instructor analytics	No	No	Heatmap and order divergence

Table 4.1: Comparison of Existing Systems and LecGap

4.3 Proposed System Architecture:

The proposed architecture is organised in four layers, as shown in Fig. 4.4. The **presentation layer** consists of two Streamlit dashboards, one for students and one for faculty, which communicate with

the back end through a thin HTTP client. The **service layer** is a FastAPI application that exposes the routes for courses, lectures, graphs, clips, quizzes and statistics, and that runs long operations as background jobs. The **pipeline layer** contains the processing modules for transcription, lecture-structure extraction, prerequisite classification, graph construction, clip cutting, quiz generation and refinement. The **data layer** stores all structured data in a relational database and the media and clips on disk.

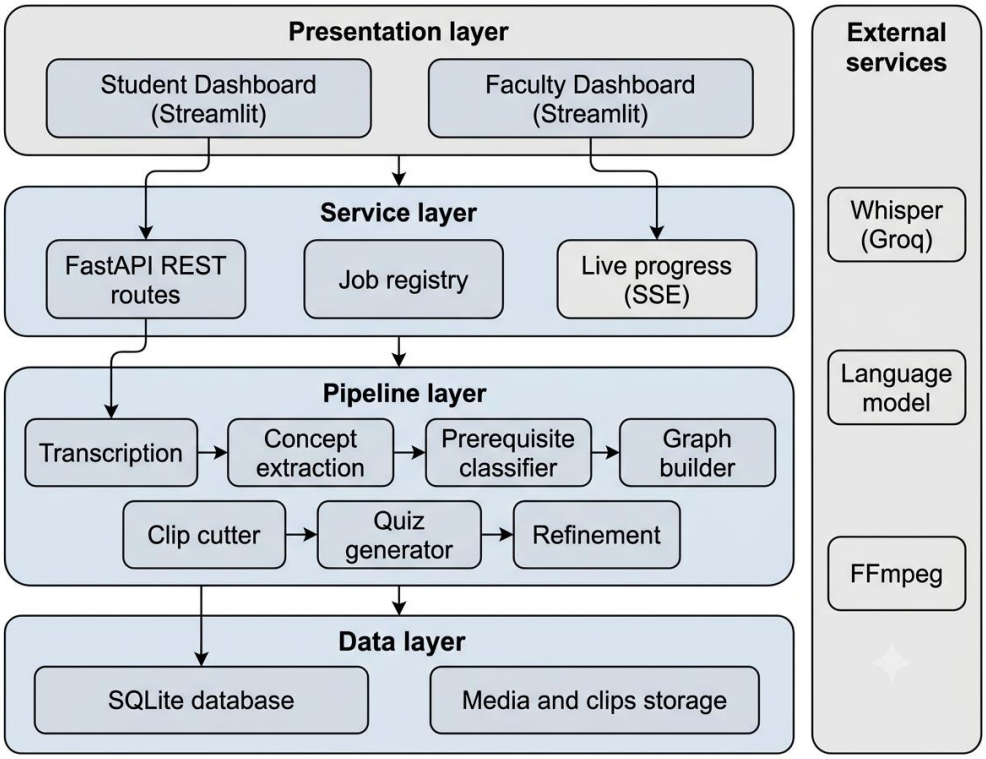


Fig. 4.4: System Architecture

Long-running work such as transcription, extraction and clip cutting never blocks the user interface. Each operation is registered as a durable job with a status, a stage and a progress percentage, and the back end streams changes to the front end using server-sent events.

Two design choices distinguish the architecture from a simple chain of models. First, prerequisites are inferred from the transcript first: when an instructor states that one concept builds on another, that statement is stored as an edge with its evidence sentence, and the classifier is used only for the remaining pairs. Second, the graph is not final: every quiz response adjusts edge confidence.

CHAPTER 5

PROJECT DESIGN & PROCESS WORKFLOW

5.1 System Flowchart Diagram:

The system flowchart describes how a lecture moves through LecGap. The recording is first transcribed. The transcript is then passed to the concept extraction stage, and the visual track, when used, contributes additional concepts from slides. The extracted concepts are checked for prerequisite relationships, merged into the course graph, and linked to video clips. When a student takes the quiz, each answer is graded; wrong answers trigger a traversal of the graph to find the root-cause prerequisite, and the resulting remediation is shown to the student. The results are also aggregated for the instructor and used to refine the graph.

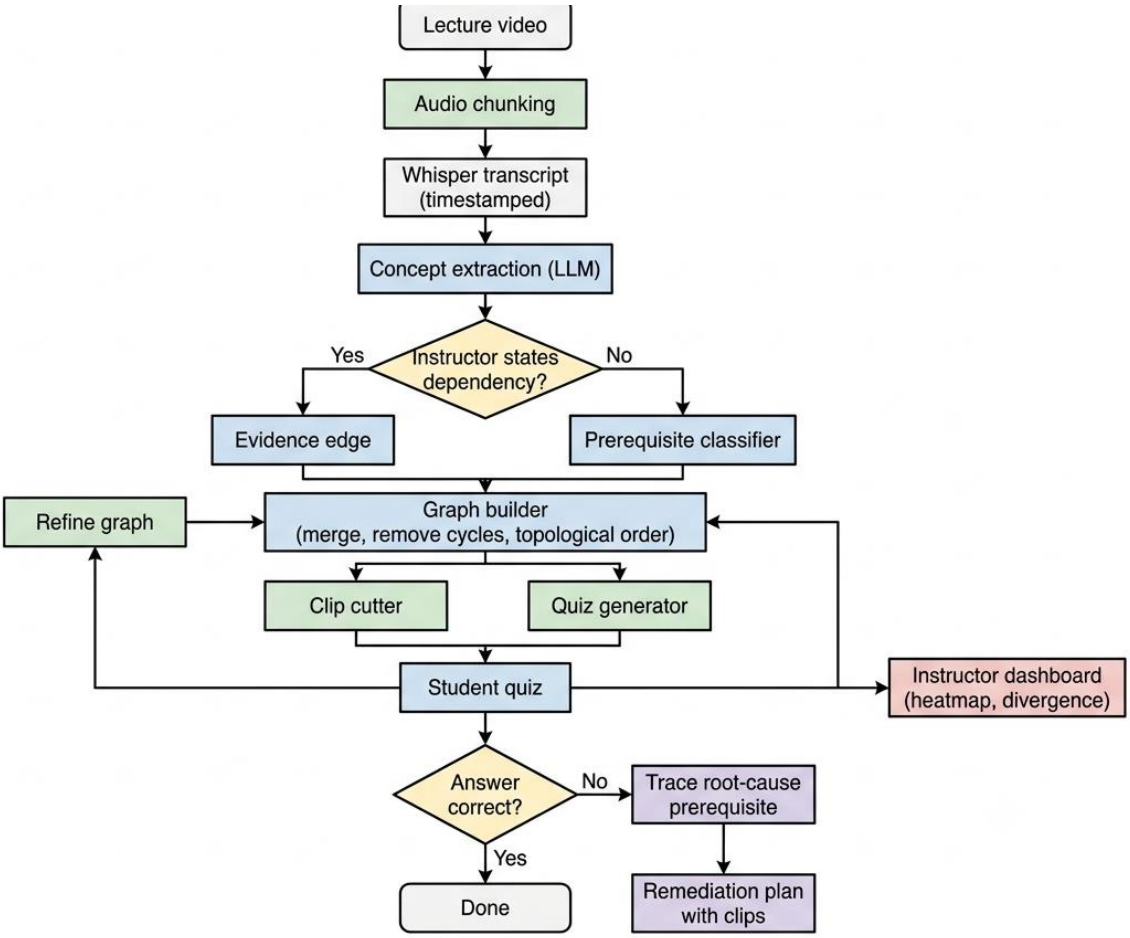


Fig. 5.1: System Flowchart

Three design principles guided the workflow. The first is modularity: each stage reads the stored output of the previous one, so it can be re-run alone. The second is caching: language model responses are stored, so repeated runs are fast. The third is transparency: every job reports its stage and progress.

5.2 Use Case Diagram:

LecGap has two human actors and one system actor. The **student** uploads or selects a lecture, takes the quiz, reads the feedback, and follows the remediation plan by watching the recommended clips in order. The **instructor** uploads lectures, starts the processing stages, inspects the concept graph, the lecture timeline and the confusion heatmap, and reviews the taught-versus-learned divergence. The **system** runs the background jobs, builds and refines the graph, grades answers on the server, and publishes progress updates.

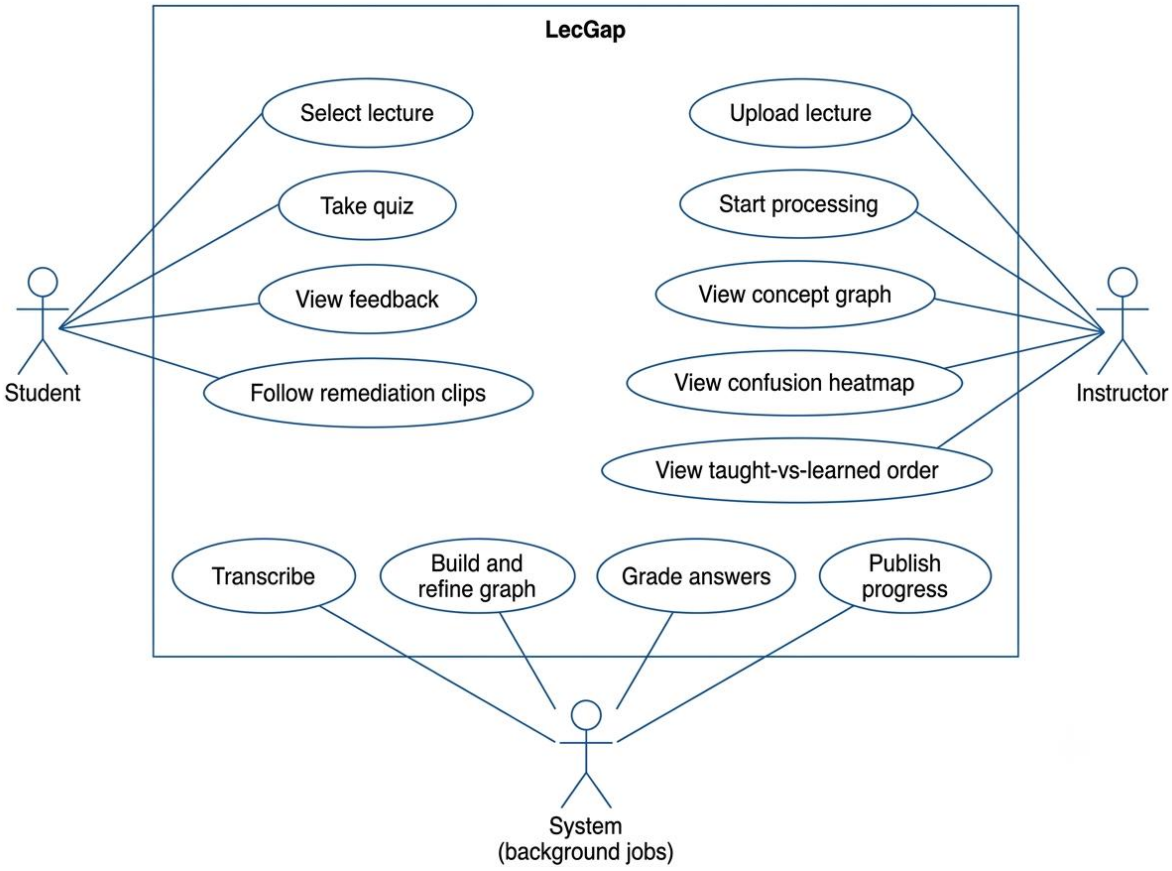


Fig. 5.2: Use Case Diagram

5.3 Algorithms:

This section describes the main algorithms used in the pipeline. Each stage is designed to be independent so that it can be tested, cached and improved on its own.

5.3.1 Transcription

Long recordings are split into audio chunks of at most five minutes, each chunk is re-encoded to avoid malformed tails, and the chunks are sent to Whisper through a hosted inference service. Failed requests are retried with back-off, and the segment timestamps are shifted back to the lecture timeline and concatenated. On measured runs this hosted path transcribes a lecture at about ten times real-time speed.

The five-minute cap and the per-chunk re-encoding were introduced after long recordings produced malformed audio tails that caused server errors. The cap keeps each request small enough to retry cheaply, and the resulting segments are stored with start and end times so that every later stage can refer back to the exact position in the lecture.

5.3.2 Lecture-Structure Extraction

Instead of treating the transcript as unrelated chunks, the system performs a lecture-structure pass. The steps are:

- Split the transcript into overlapping windows of segments that fit within the model's context
- Ask the language model to return, for each window, the passages in which a concept is taught, the concept name and the spoken links between concepts
- Repair or salvage the model's JSON output when it is truncated or contains invalid escape sequences, so that complete passages are never lost
- Store the passages, concepts and links, and anchor each concept to the time span in which it is actually taught

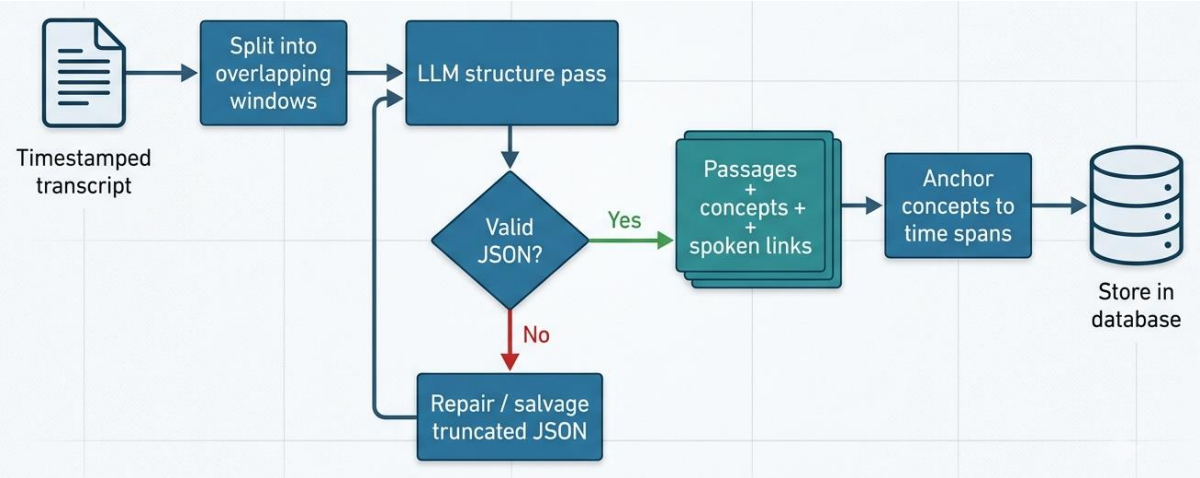


Fig. 5.3: Concept Extraction Flowchart

Windows overlap so that a concept explained across a window boundary is not cut in two, and later windows see the end of the previous one. Because every concept is stored with its start and end times, the same record drives the video clips, the timeline view and the quiz questions.

5.3.3 Prerequisite Classification

For concept pairs that are not explained by transcript evidence, a classifier decides whether one concept is a prerequisite of the other. Concept names are embedded using a compact Sentence-BERT-style encoder. The feature vector for a pair combines the two embeddings with their absolute difference, their element-wise product and their cosine similarity. A logistic classifier is trained on LectureBank using negative undersampling, and the decision threshold is chosen on a held-out validation slice so that the test data is never used for tuning. A frozen encoder has a practical advantage: it needs no GPU at inference time, and the same embeddings are reused for duplicate merging in the graph builder.

Cross-encoder fine-tuning was also evaluated on a Kaggle GPU with several grids of epochs, learning rates and negative ratios, and with a larger MPNet backbone. Cross-validated F1 scores for the fine-tuned models ranged from 0.49 to 0.55, all below the frozen-encoder baseline of 0.569. A shipping gate was therefore applied, and the frozen-encoder classifier was adopted as the baseline model.

5.3.4 Graph Construction

The concept graph is built per course. Concept names are embedded and near-duplicates are merged into a single canonical node, with a map from the raw names to the canonical names. Edges from transcript evidence are inserted with high confidence and an evidence sentence, and classifier edges are added for the remaining pairs. Where the same edge is proposed more than once, the highest

confidence is kept. Self-loops created by merging are dropped. Cycles are then removed by repeatedly finding a cycle and deleting its lowest-confidence edge, until the graph is acyclic. A topological sort finally gives a valid learning order. The merge threshold is deliberately conservative, because wrongly merging two distinct concepts is more harmful than keeping two near-duplicate nodes.

5.3.5 Quiz Generation and Remediation

For each concept, the system builds a multiple-choice question from the lecture itself. The stem quotes or paraphrases the passage in which the concept is taught, and the distractors are drawn from other concepts. The correct answer is kept on the server, and grading is performed there so that it cannot be read from the page. When a student submits answers, each wrong concept is traced back through its prerequisites in the graph. Concepts that the student has also failed, or that are the earliest unmet prerequisites, are placed first in the remediation plan, and the matching video clips are attached in that order.

Question stems are built from the lecture's own sentences so that they test what was actually taught rather than general knowledge, and the distractors come from other concepts of the same lecture, which keeps them plausible. The number of questions is capped to keep the quiz short, and the cap can be configured.

As an illustration, suppose a student answers a question on backpropagation incorrectly. The system looks up the node for backpropagation, follows its incoming edges to gradient descent and loss functions, and checks the student's answers on those nodes. If gradient descent was also answered incorrectly, it is placed before backpropagation in the plan, and the clip for each concept is attached

so that the student can repair the earliest gap first and then return to the concept they originally missed.

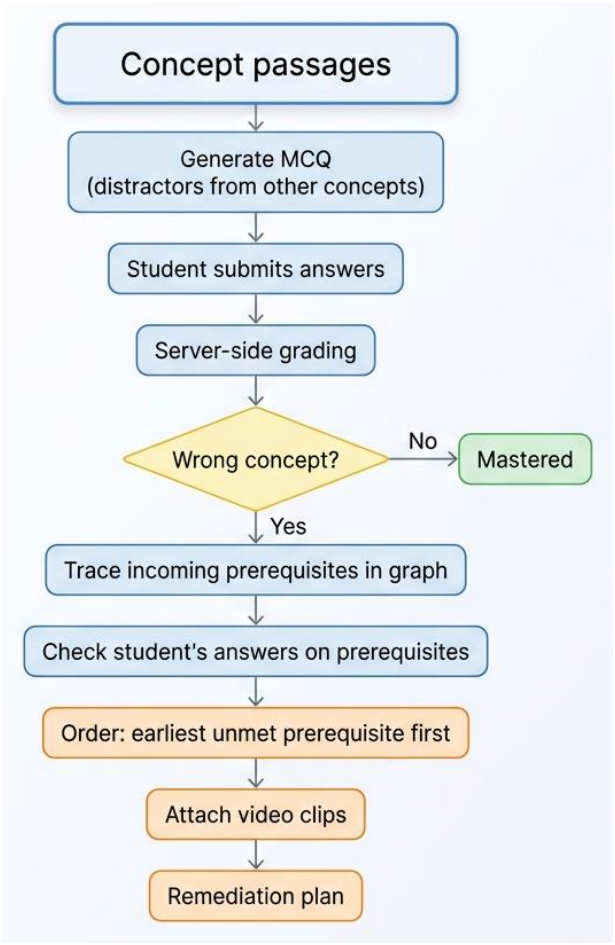


Fig. 5.4: Quiz and Remediation Flowchart

Table 5.1 shows how the remediation order is derived in the example that follows, for a student who answers the backpropagation question incorrectly.

Order	Concept	Student result	Remediation
1	Loss functions	Correct	No action
2	Gradient descent	Incorrect	Revise first: clip on gradient descent
3	Backpropagation	Incorrect	Revise second: clip on backpropagation
4	Neural networks	Correct	No action

Table 5.1: Example of Remediation Order

5.3.6 Graph Refinement by Weak Supervision

The refinement step uses a co-failure rule over many students. If students who fail concept A also tend to fail concept B, the edge from A to B is reinforced. If students fail A but not B, the edge is weakened. Over many responses, spurious edges sink and genuine edges are strengthened. The mechanism was validated with simulated students: with 200 structural personas the F1 score of the graph rose from 0.571 to 0.857 with a precision of 1.0, and with eight language-model personas that genuinely attempted the quiz it rose from 0.571 to 0.750. These are simulated results and are not a claim about live classrooms.

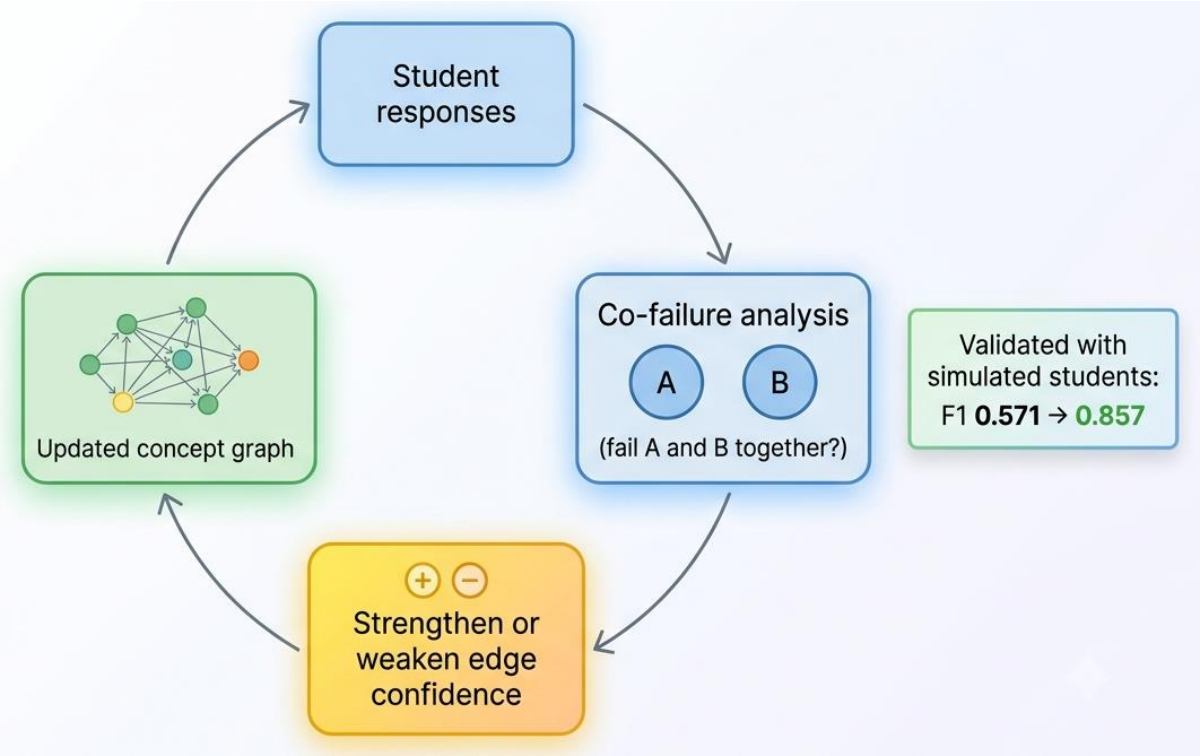


Fig. 5.5: Refinement Loop Flowchart

The main limitation of this experiment is that simulated students answer according to rules that the experimenter defines, so the result shows that the mechanism can recover a correct graph under controlled conditions rather than that it will do so with real learners. Validation on a live class is therefore listed as future work.

The refinement runs after each batch of responses and never deletes a transcript-derived edge without evidence from several students, so a single unusual student cannot damage the graph. Because the rule only needs the pattern of correct and incorrect answers, it can run on the data that the quiz already collects and requires no extra effort from students or instructors.

Graph quality in these experiments is measured against the reference prerequisite pairs using precision, recall and F1. Precision is the fraction of proposed edges that are correct, recall is the fraction of true edges that are found, and F1 is their harmonic mean, so a graph scores well only if it is both accurate and complete. The same metrics are used to compare the frozen-encoder baseline with the fine-tuned models described in Section 5.3.3.

5.4 Software Specifications:

The software stack consists entirely of open-source tools and hosted inference services. Table 5.2 lists the components and their roles.

Component	Technology	Purpose
Programming language	Python 3.10	Implementation of back end, pipeline and dashboards
Back-end framework	FastAPI with Uvicorn	REST API, background jobs, server-sent events
Front end	Streamlit (React rebuild planned)	Student and faculty dashboards
Speech recognition	Whisper (hosted on Groq)	Timestamped transcription
Language model	Hosted large language model	Concept extraction and quiz writing
Embeddings	Sentence-BERT-style encoder (MiniLM)	Concept similarity and classifier features
Graph library	NetworkX	Graph construction, cycle removal, ordering
Media processing	FFmpeg	Audio chunking and clip cutting
Database	SQLite with SQLAlchemy	Courses, lectures, concepts, graph, quizzes, jobs
Visualisation	PyVis and custom HTML	Concept graph and lecture timeline
Testing	pytest and Streamlit AppTest	Automated tests of back end and dashboards
Training environment	Kaggle notebooks (GPU)	Fine-tuning experiments

Table 5.2: Software Requirements

5.5 Hardware Specifications:

LecGap is designed so that the heavy language and speech models run on hosted services, which keeps the local hardware requirements modest. A GPU is needed only for the optional fine-tuning experiments, which were run on a free cloud GPU. Table 5.3 lists the recommended configuration.

Component	Specification	Remarks
Processor	Quad-core CPU or better	Runs the back end, encoder inference and FFmpeg
Memory	8 GB RAM minimum, 16 GB recommended	Needed to load the encoder and process long videos
Storage	At least 20 GB of free disk space	Raw lectures and generated clips are large
GPU	Optional; Tesla T4 class on Kaggle	Only for fine-tuning experiments
Network	Stable broadband connection	Required for hosted transcription and language model calls
Display	Standard monitor and a modern web browser	Used to access the dashboards

Table 5.3: Hardware Requirements

5.6 User Requirements:

The requirements below were derived from the needs of the two user groups.

- **Students** need a short quiz for each lecture, immediate and understandable feedback, and a clear order in which to revise, with short clips rather than the full recording
- **Instructors** need to upload a lecture and see its progress, to inspect the concept graph and the evidence behind each edge, and to see which concepts confuse the class and whether the teaching order matches the dependencies
- **Both groups** need an interface that stays responsive during long operations, shows clearly whether a job is running or finished, and reports errors in plain language
- **Administrators** need the system to protect its data: uploads are size-limited, file names are sanitised, answer keys are never sent to the client, and access can be restricted with an API key

In addition to these needs, the system is expected to satisfy the following functional and non-functional requirements.

- **Functional:** upload a lecture and process it into a transcript, concepts and a graph; generate a quiz for each lecture; grade answers on the server; produce an ordered remediation plan with clips; and display analytics to instructors
- **Reliability:** a failed or interrupted job must be visible and recoverable, and a repeated request must not corrupt data or block other work
- **Performance:** transcription should run much faster than real time, and the interface should never freeze while a job is running
- **Usability:** the student view should need no training, and every screen should state clearly what the user can do next
- **Security:** uploads are validated, answer keys stay on the server, and administrative routes can be protected with an API key
- **Maintainability:** each pipeline stage is a separate module with automated tests, so that components can be replaced without rewriting the system
- **Portability:** the system runs on an ordinary laptop and relies on hosted services only for the heavy speech and language models

Together, these requirements define the acceptance criteria for the Semester VIII evaluation: a lecture must be processed end to end without manual intervention, a student must receive a correct and ordered remediation plan, and an instructor must be able to read the class-level analytics without technical help.

5.7 Chapter Summary:

This chapter presented the design of LecGap in detail. The main points are:

- **Workflow:** a lecture flows from transcription through concept extraction and graph building to quizzes, remediation and instructor analytics
- **Actors:** students, instructors and the system itself each have clear use cases
- **Algorithms:** six stages, from chunked transcription to graph refinement, each stored and testable on its own
- **Specifications:** open-source software, hosted inference and modest local hardware
- **Requirements:** reliability, performance, usability, security, maintainability and portability define acceptance for Semester VIII

These design choices were exercised on real lecture recordings, and the resulting findings and the limitations of the present baseline are discussed in Chapter 6.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

Conclusion:

This project set out to make recorded lectures diagnostic, so that a wrong answer leads a student to the real gap in understanding instead of leaving them to search a long video. The result is LecGap, a working baseline that transcribes a lecture, extracts the concepts that are taught, builds a course-level prerequisite graph, cuts short concept clips, grades a quiz on the server, and returns a remediation plan ordered by prerequisite dependency. The same data gives instructors a confusion heatmap, a lecture timeline and a comparison between the taught order and the learned order.

The work also produced clear technical findings. Transcript evidence is a stronger source of prerequisites than a name-only classifier, and the frozen-encoder classifier with an F1 score of 0.569 outperformed every fine-tuned variant that was tried, so it was kept as the fallback model. A co-failure refinement rule improved the graph in simulation, from an F1 of 0.571 to 0.857 with 200 simulated students. The pipeline was run end to end on real lectures of 21 to 86 minutes, and engineering work on job tracking, live progress, security and testing has made the system stable enough to demonstrate. The refinement results are simulated and still need to be confirmed with real students.

In summary, LecGap shows that a recorded lecture can be turned into a structured, self-correcting model of what must be learned and in what order, using only open tools and hosted services. The baseline is a sound foundation for the planned evaluation with real students.

Limitations:

The work reported here has several limitations that should be kept in mind when interpreting the results.

- **Simulated refinement:** the improvement from 0.571 to 0.857 was measured with simulated students, so it demonstrates the mechanism rather than classroom impact
- **Small benchmark:** LectureBank covers a limited number of topics, mainly in natural language processing, so the classifier may transfer imperfectly to other subjects
- **Speech-only extraction:** concepts shown only on slides or whiteboards are not yet captured, because the visual track is still under development
- **Hosted dependencies:** transcription and language-model steps rely on external services, so cost, latency and availability are outside the control of the project
- **Single-machine deployment:** the baseline runs on one machine and has not been load-tested with many concurrent users

Future Scope:

Several directions remain for Semester VIII and beyond.

- **React front end:** replace the Streamlit interface with a React application that shows live job progress, keeps video playback undisturbed, and offers a cleaner layout
- **Job-queue hardening:** add periodic recovery of stalled jobs, bounded waiting for pipeline slots, and per-lecture locks so that duplicate requests cannot block the system
- **Visual track:** complete the extraction of concepts from slides and whiteboards using CLIP and OCR, so that concepts that are shown but not spoken are captured
- **Evaluation with real students:** run the system on a live class and compare the refined graph and remediation plans with instructor judgement and with student learning outcomes
- **Wider coverage:** support more subjects and languages, and test the cross-domain transfer of the prerequisite classifier
- **Scalability:** move from a single-machine database to a multi-user deployment with authentication, role-based access and monitoring

Together, these steps will move LecGap from a working baseline to a system that can be evaluated in a real classroom.

Final Remarks:

LecGap began with a simple question: how can a recorded lecture tell a student not only that an answer is wrong, but why? The project answers it by treating a lecture as a structured set of concepts that depend on one another, and by using that structure to turn every mistake into a specific, ordered recommendation. The same structure gives instructors an honest view of how well a lecture was understood.

The baseline built in Semester VII demonstrates that the idea is workable with open tools and hosted services. The remaining work is mainly about evidence and polish: confirming the refinement results with real students, improving the interface, and hardening the system for wider use.

The project has also been a practical exercise in applied machine learning engineering: benchmarking a model before adopting it, treating failures as data, keeping every stage separately testable, and reporting results honestly, including the ones that did not improve on the baseline.

References

[1] I. Li, A. R. Fabbri, R. R. Tung and D. R. Radev, "What Should I Learn First: Introducing LectureBank for NLP Education and Prerequisite Chain Learning," in Proc. AAAI Conference on Artificial Intelligence, 2019.

[2] I. Li, V. Yan and D. Radev, "Unsupervised Cross-Domain Prerequisite Chain Learning using Variational Graph Autoencoders," in Proc. Annual Meeting of the Association for Computational Linguistics (ACL), 2021.

[3] L. Pan, C. Li, J. Li and J. Tang, "Prerequisite Relation Learning for Concepts in MOOCs," in Proc. Annual Meeting of the Association for Computational Linguistics (ACL), 2017.

[4] A. T. Corbett and J. R. Anderson, "Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge," User Modeling and User-Adapted Interaction, vol. 4, no. 4, pp. 253–278, 1994.

[5] C. Piech et al., "Deep Knowledge Tracing," in Advances in Neural Information Processing Systems (NeurIPS), 2015.

[6] A. Radford et al., "Robust Speech Recognition via Large-Scale Weak Supervision," in Proc. International Conference on Machine Learning (ICML), 2023.

[7] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," in Proc. International Conference on Machine Learning (ICML), 2021.

[8] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP), 2019.

[9] A. A. Hagberg, D. A. Schult and P. J. Swart, "Exploring Network Structure, Dynamics, and Function using NetworkX," in Proc. 7th Python in Science Conference (SciPy), 2008.

[10] A. Vaswani et al., "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[11] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL-HLT, 2019.

[12] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[13] S. Ramírez, "FastAPI documentation," https://fastapi.tiangolo.com.

[14] Streamlit Inc., "Streamlit documentation," https://docs.streamlit.io.

Publications

[To be updated: details of any research paper, poster, or competition entry resulting from this project, including title, venue and date.]